

Mini Paper — 1.2 Software & Software Development — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Q1 (2 marks). - Using secondary storage / part of the hard disk (page/swap file) as if it were RAM (1). - To allow programs larger than physical RAM to run / to allow more programs to be open at once than would fit in RAM (1).

Examiner tip: Do not write that it "adds more RAM" — it uses slower secondary storage as if it were RAM.

Q2 (3 marks). - Paging divides memory into **fixed-size** blocks (pages/frames) (1). - Segmentation divides memory into **variable-size** blocks (segments) based on the program's logical structure (1). - Paging suffers from **internal** fragmentation; segmentation suffers from **external** fragmentation (1).

Examiner tip: The reliable discriminator is fixed-size (paging) vs variable/logical (segmentation) — state it explicitly.

Q3 (4 marks). 1 mark each, max 4: - At the end of the current fetch–decode–execute cycle the CPU checks for interrupts (and completes the current instruction if one of sufficient priority is present). - The contents of the registers (PC, accumulator, etc.) are saved/pushed onto the **stack**. - The CPU loads and runs the appropriate **interrupt service routine** to deal with the interrupt. - When the ISR finishes, the saved register contents are popped/restored from the stack and the interrupted task resumes where it left off.

Examiner tip: Forgetting the **stack** (so the original job can resume) is the most common lost mark.

Q4 (4 marks). (a) - Application software lets the user perform a specific real-world task; utility software maintains/configures/optimises the computer itself (1). - One valid example of each, e.g. application = web browser/word processor; utility = antivirus/defragmenter/backup (1).

(b) - **Embedded operating system** (1). - Justification: the thermostat is a dedicated device with limited resources; an embedded OS is small, efficient and held in ROM to control one specific task reliably (1).

Examiner tip: Antivirus is a **utility**, not an application — keep "maintains the computer" vs "does the user's task" clear.

Q5 (4 marks). 1 mark each, in order, max 4: - **Lexical analysis** — source code broken into tokens; whitespace/comments removed; symbol table created. - **Syntax analysis (parsing)** — tokens checked against the language's grammar; parse tree built; syntax errors reported. - **Code generation** — machine/object code produced from the parse tree and symbol table. - **(Code) optimisation** — code refined to run faster / use less memory while giving the same result.

Examiner tip: Keep order correct and don't muddle lexical (tokens) with syntax (grammar). Grammar errors are found in syntax analysis.

Q6 (4 marks). - (a) **Immediate = 40** — the operand itself is the value loaded (1). - (b) **Direct = 25** — the operand is an address; the contents of address 40 (= 25) are loaded (1). - (c) **Indirect = 9** (1) — address

40 holds 25, which is itself an address, so the contents of address 25 (= 9) are loaded (explanation that it is the "address of an address") (1).

Examiner tip: Indirect = "address of an address"; trace carefully through the memory table given.

Q7 (3 marks). - Encapsulation = bundling attributes and methods together within a class and hiding the data, so private attributes are accessed only through methods (information hiding) (1 for bundling, 1 for data hiding — max 2). - Maintainability/robustness: the internal implementation can change without breaking other code / invalid values can be rejected by getters and setters, protecting the object's state (1).

Examiner tip: For full marks, link data hiding to a concrete benefit (validation or being able to change implementation safely).

Q8 (6 marks). Indicative content; reward application and evaluation, not description alone.

Recommendation — **Agile** (RAD also creditable): - Requirements are uncertain/likely to change, which suits an **iterative** approach rather than a fixed linear plan (1). - Founders want a usable release quickly — agile delivers **working increments early and often** (1). - They want to see and respond to progress regularly — agile gives continuous client feedback and lets changes be incorporated cheaply between iterations (1).

Evaluation of an alternative: - **Waterfall** rejected — it is linear/inflexible, fixes requirements up front (which the start-up does not have), and the client only sees the product at the end (1). - **Spiral** rejected — its heavy, expensive risk analysis is overkill for a small, limited-budget app (1). - A reasoned conclusion that ties the choice back to the specific scenario clues (uncertain requirements, speed, feedback, limited budget) (1).

Mark bands: - **5–6:** clear justified recommendation applying *specific scenario clues*, with evaluation of an alternative and a reasoned conclusion. - **3–4:** sound recommendation with some application/justification; limited evaluation. - **1–2:** mainly description of methodologies with little application to the scenario.

Examiner tip: This is AO3 — you must *apply and justify* against the scenario's clues; a pure description of agile caps your marks.